

High-Capacity Framework for Reversible Data Hiding Using Asymmetric Numeral Systems

Na Wang, Shuxi Xu, Chuan Qin, Sian-Jheng Lin, *Member, IEEE*, Shuo Shao, and Yunghsiang S. Han, *Fellow, IEEE*

Abstract—Reversible data hiding (RDH) has been extensively studied in the field of multimedia security. Embedding capacity is an important metric for RDH performance evaluation. However, the embedding capacity of existing methods for independent and identically distributed (i.i.d.) gray-scale signals is still not good enough. In this paper, we propose a high-capacity RDH code construction method that employs asymmetric numeral systems (ANS) coding as the underlying coding framework. Based on the proposed framework, two RDH methods are presented. First, we propose a static RDH method that takes the constant host probability mass function (PMF) as input parameters and offers high embedding performance. Then, we give a dynamic RDH method that can eliminate the need for transmitting the host PMF in advance by designing a reversible dynamic probability calculator. The simulation results on discrete normally distributed signals demonstrate that the performance of the proposed static method is very close to the expected rate-distortion bound, and the proposed dynamic method can achieve satisfactory embedding capacity without prior knowledge of host PMF at the cost of slightly sacrificing steganographic data quality. Moreover, the experimental results on gray-scale images show that the proposed static method provides higher peak signal-to-noise ratio (PSNR) values and larger embedding capacities than some state-of-the-art methods, e.g., the embedding capacity of image Lena is as high as 3.571 bits per pixel.

Index Terms—Entropy coding, Asymmetric numeral systems, Reversible data hiding, Embedding capacity, Gray-scale signals.

1 INTRODUCTION

REVERSIBLE data hiding (RDH) embeds secret data into communication media to achieve secure transmission. A media used to embed data is termed host media, and the media embedded with secret data is called steganographic media (abbreviated stego media). RDH has a special property that the receiver can losslessly reconstruct host media and secret data from the received stego media. It is widely used in medical imagery [1], ownership protection [2] and law forensics, where the host media is so precious that it cannot be destroyed. For example, RDH can be used to protect the copyright of relational databases [3–7].

RDH work can be categorized mainly into the plain domain and the encrypted domain. RDH methods in the plain domain focus on techniques to enhance payload capacity while maintaining low distortion. In this domain, secret data is embedded into the host media without encrypting the media itself. On the other hand, due to its potential applications in cloud computing as privacy protection, RDH in the encrypted domain is extensively explored. In this domain, the host media is encrypted, and secret data is embedded into the encrypted media without revealing the

plaintext data. In the past decades, numerous RDH methods in the plain domain have been proposed [8–19]. Early RDH schemes are mainly based on lossless compression, where the embedding space is created by losslessly compressing some features of the host media [8]. However, because of the weak correlation in the bit plane (i.e., a layer of binary data that represents a specific bit position within the binary representation of each signal), lossless compression-based RDH methods have a limited embedding capacity, where the embedding capacity is the size of the secret data payload that can be embedded in the host media. Later on, the plain domain work is roughly divided into two categories: correlation expansion [9, 10] and histogram shifting [15, 16, 18, 19]. The basic idea behind the correlation expansion technique for RDH is to subtly modify the correlation between adjacent or nearby data elements in the host media to embed additional information without significantly altering the original signal. Similarly, histogram shifting-based RDH techniques modify the distribution of pixel intensity values in the host media in a reversible manner to embed additional information without causing noticeable distortion. Also, correlation work is further classified as difference expansion (DE) and prediction error expansion (PEE). Both DE and PEE are techniques used in RDH, but they differ in their approach to modifying the signal for embedding additional data. In DE, modifications are directly applied to the differences between adjacent elements in the host media. PEE involves predicting the value of each element based on its neighboring elements and then expanding the prediction errors to embed data. The correlation expansion approach utilizes the correlation of spatial domain for hiding secret information in the host media, while in histogram shifting, the peak of the histogram bin is used. Zhang et al. [11]

- Na Wang and Chuan Qin are with the School of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology, Shanghai 200093, China. E-mail: wna@usst.edu.cn; qin@usst.edu.cn.
- Shuxi Xu is with the East China Institute of Optoelectronic, Integrated Devices, Bengbu 233000, China. E-mail: laoer51983193@163.com.
- Sian-Jheng Lin is an independent researcher in China. E-mail: sjhenglin@gmail.com. (Corresponding author: Sian-Jheng Lin.)
- Shuo Shao is with the School of Electronic Information and Electric Engineering, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: shuoshao@sjtu.edu.cn.
- Yunghsiang S. Han is with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518110, China. E-mail: yunghsiangh@gmail.com.

considered both pixel value orders and pixel locations so that the generated prediction-error histogram (which shows the frequency of occurrence of different prediction error values in the image) is regular in shape and suitable for reversible embedding. In [12], He et al. introduced an effective RDH scheme based on dual pairwise PEE. Kumar et al. [20] gave a high-fidelity RDH scheme based on local moment-driven pixel value ordering. In [13], an RDH scheme using adaptive pixel value prediction and adaptive embedding bin selection was proposed. By considering the differences between adjacent pixels rather than simple pixel values, Tai et al. [14] presented an efficient extension of the histogram modification technique, which involves altering the distribution of intensity values or samples in the host's histogram to embed data. Kim et al. [15] adopted skewed histograms with a pair of extreme predictions to achieve RDH. In [16], an adaptive modification for multiple prediction-error histograms (PEHs) was proposed by considering the image content. Roy et al. [17] proposed a watermarking system based on the Discrete Cosine Transform (DCT) [21], which transforms data points in the spatial domain into the frequency domain to make it easier to find the repetition of patterns, by utilizing the psycho-visual threshold standard. In [18], Ou et al. extended the multiple histogram modification scheme and proposed a high-capacity embedding method by exploiting multiple pairs of bins for expansion in each PEH. Multiple histogram modification and pairwise PEE can significantly reduce the embedding distortion of the host media for low and medium embedding payloads. However, the histogram-based approaches only hide data in smooth regions of the host image and meanwhile, discard other ones, thus limiting its embedding capacity.

Embedding capacity is an important metric for RDH performance evaluation, and certain practical applications prefer high embedding capacity. For example, in the secret image sharing field, in order to embed a given secret image, the larger the embedding capacity of the scheme is, the smaller size of host images is needed, which results in less time and space being demanded during transmission and storage. Therefore, in this paper, we focus on the RDH methods with high embedding capacity. Some existing RDH algorithms also aim to reach desirable embedding capacity. For the i.i.d. host sequence, Kalker and Willems [22] formulated the RDH as a rate-distortion problem and derived the upper bound (rate-distortion bound) of the embedding capacity under a given distortion constraint. With this derivation, some practical code construction methods [22–27] were proposed to asymptotically approach the rate-distortion bound. For the binary i.i.d. case, Kalker et al. [22] provided a recursive code construction (RCC), and Zhang et al. [23] improved the RCC by designing a more efficient compression algorithm. In [28], a general distortion model was formulated for binary host images. Indeed, gray-scale signals (8-bit) are more general than binary signals in practical applications. Thus, this paper concentrates solely on the gray-scale host signals. For the gray-scale i.i.d. case, Zhang et al. [25] extended the RCC to various applications. In [26], Zhang et al. implemented a novel RCC for RDH by recursively utilizing the decompression and compression processes of arithmetic coding [29]. Although the RCC-based RDH scheme can achieve encouraging results, its

embedding performance is still not satisfactory when a large embedding capacity is required. In [27], Lin et al. proposed the first RDH scheme that asymptotically approaches the theoretical rate-distortion bound by using arithmetic coding. Arithmetic coding is a lossless data compression technique that encodes data as a single real number between 0 and 1. It maps a string of symbols to a fractional number with more probable symbols represented by larger fractional ranges. The encoding of arithmetic coding involves repeatedly dividing the interval based on symbol probabilities, and the final encoded number represents the entire string. A fundamental problem with arithmetic coding is that it involves a lot of arithmetic, and even after slightly degrading coding effectiveness in order to use only single-word fixed-precision rather than arbitrary-precision arithmetic, state-of-the-art implementations are still complicated and relatively slow. In particular, because of the usage of arithmetic coding, the coding interval in Lin et al.'s implementation is slightly narrower than the previous interval (see [27] for more details), which will slightly decrease the embedding capacity. When the coding interval is too small, further embedding is not possible. Therefore, to prevent the above situation, the authors use dynamic precision numbers to ensure that the precision is enough to represent the coding interval. However, when the precision becomes too large, it imposes high computational demand. Moreover, the extraction procedures in [25–27] require prior knowledge of the host probability distribution, which shall also be transmitted to the receiver to help with decoding. This requirement results in additional storage space overhead and is not suitable for memory-constrained environments, such as wireless sensor networks. Also, there is usually no other channel for transmitting additional data in the generic data hiding model. Additionally, in some telecommunication systems, receivers may not have access to the required host statistics. For instance, Duda et al. [30, 31] proposed a novel method for generating image-like 2D barcodes with the desired statistical properties that are known only to the sender. Therefore, RDH methods that do not rely on this pre-transmission requirement are more practical.

In this paper, we propose a high-capacity code construction method, which employs the asymmetric numeral systems (ANS) coding [32, 33] (see Section 2.2.2 for details) as the underlying RDH coding framework. ANS coding is a new approach to precise entropy coding. It is as fast as Huffman coding and nearly as accurate as arithmetic coding. Unlike arithmetic coding, which requires two variables to represent a range, ANS coding uses a single fixed-precision natural number as the state. In addition, ANS coding employs faster encoding and decoding algorithms and typically requires less computational resources than arithmetic coding. Our coding framework consists of a mixture of ANS encoder and decoder, where the encoder is designed to embed secret data, the decoder produces the corresponding stego signals, and the output of the ANS encoder acts as temporary information to communicate with the ANS decoder. The reasons for using ANS coding in RDH are as follows. Firstly, the ANS coding operates on positive integers with fixed precision, and therefore, it does not suffer from the computing precision problem that may occur in arithmetic coding. Secondly, it has been stated in

[33] that the coding efficiency of ANS coding can approach the Shannon entropy [34], which quantifies the average amount of information produced by a stochastic process or received through a communication channel per unit of time or symbol. These above observations motivate our work. Notably, because the ANS encoder and decoder run in opposite directions, i.e., the order of symbols processed by the encoder is the reverse of the order of symbols reconstructed by the decoder, our proposed framework is totally different from the traditional RDH code construction methods, and the classical arithmetic coding based constructions cannot be applicable. Based on the proposed framework, two RDH methods are presented. Among them, one static RDH method requires the host probability mass function (PMF) like existing methods [25–27] but exhibits high embedding performance, and the other dynamic RDH method removes the need to transmit the host PMF, thus saving the space overhead. To the best of our knowledge, this paper is the first to utilize ANS coding for RDH.

The main contribution of this paper is to propose two RDH methods that employ ANS coding as the underlying framework with the host PMF delivered or omitted, respectively. Specifically, the contributions in this paper are as follows.

- 1) A static RDH method that takes the host PMF as input parameters but offers high embedding capacity is proposed. Numerical results on discrete normally distributed signals show that the performance of the proposed static method is very close to the expected rate-distortion bound. Moreover, for gray-scale images, the proposed static method provides higher PSNR values and larger embedding capacities than some state-of-the-art methods, e.g., the embedding capacity of image Lena is as high as 3.571 bits per pixel.
- 2) In order to avoid the need to transfer the host PMF in advance, a dynamic RDH method is proposed, which designs a reversible dynamic probability calculator to use a varying PMF instead of the static PMF. The experimental results show that the proposed dynamic method saves overhead and achieves satisfactory embedding capacities at the cost of a small stego quality loss.

The rest of this paper is organized as follows. Section 2 lists the notations and related works. Section 3 introduces the proposed static RDH method. In Section 4, we give a dynamic RDH method. Section 5 shows the simulation results, and Section 6 concludes this work.

2 PRELIMINARIES

2.1 Notations

In RDH, L bits of message $W = (w_1, w_2, \dots, w_L)$ are embedded by the embedder into host sequence $X = (x_1, x_2, \dots, x_N)$ by slightly modifying the elements of X to produce a stego sequence $Y = (y_1, y_2, \dots, y_N)$, where $w_i \in \{0, 1\}$ and $x_j, y_j \in \{0, 1, \dots, B-1\}$ for $1 \leq i \leq L$, $1 \leq j \leq N$ and $B \in \mathbb{N}^+$. The extractor can losslessly reconstruct the message W and the host sequence X from the stego sequence Y . That is, data extraction involves extracting the host sequence X and embedded message W from the received stego sequence Y . Let $R = L/N$ denote the embedding capacity. Figure 1 shows

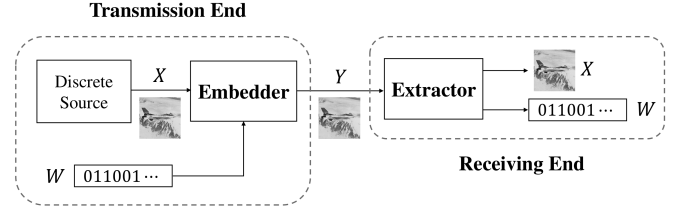


Fig. 1: An RDH schematic on a gray-scale image.

an RDH schematic on a gray-scale image X , where Y is the stego image with some distortion. For the i.i.d. host signals, Kalker and Willems [22] formulated the upper bound R_{max} of the embedding capacity under a given distortion constraint Δ as follows:

$$R_{max} = \text{maximize}_{P_{Y|X}(y|x)} \{H(Y)\} - H(X), \quad (1)$$

where X and Y denote the random variables of host media and stego media, respectively, and $H(\cdot)$ is the Shannon entropy function. The maximum entropy in (1) is over all transition probability matrices $P_{Y|X}(y|x)$ satisfying the distortion constraint

$$\sum_{x \in X, y \in Y} P_X(x) P_{Y|X}(y|x) D(x, y) \leq \Delta, \quad (2)$$

where $P_X(x)$ is the probability of host signal x , and the distortion metric $D(x, y)$ in this paper is defined as the square error distortion, i.e., $D(x, y) = (x - y)^2$. This paper focuses on the practical RDH code construction methods that asymptotically approach the rate-distortion bound in (1) to achieve high embedding capacity.

Throughout this paper, the host sequence $X = (x_1, x_2, \dots, x_N)$ is composed of i.i.d. samples drawn from the PMF $P_X = \{P_X(x) | x \in \Sigma_B\}$, where $P_X(x)$ is the probability of host signal x , and the set $\Sigma_B = \{0, 1, \dots, B-1\}$ is an alphabet of size B . This paper considers embedding binary data into 8-bit gray-scale signals, so $B = 256$. Let $P_{CX} = \{P_{CX}(k) = \sum_{z=-1}^{k-1} P_X(z) | k \in \Sigma_{B+1}\}$ denote the cumulative PMF of sequence X , where $\Sigma_{B+1} = \Sigma_B \cup \{B\}$ and $P_X(-1) = 0$, thus we have $P_{CX}(B) = 1$. Let $f_X(x)$ denote the number of occurrences (i.e., frequency) of signal x in the sequence X , and $CF_X = \{cf_X(k) = \sum_{z=-1}^{k-1} f_X(z) | k \in \Sigma_{B+1}\}$ is the cumulative frequency distribution of sequence X , where $f_X(-1) = 0$, so we have $cf_X(0) = 0$ and $cf_X(B) = N$.

Given a binary stream $M = (m_0, m_1, \dots, m_{\ell-1})$, we construct a LIFO (Last In First Out) stack accordingly. The operation

$$Q \leftarrow Pop_M(i) \quad (3)$$

pops $i \leq \ell$ bits $\{m_j\}_{j=0}^{i-1}$ from the top position in the stack that contains M , and these i bits form an integer $Q = m_0 \times 2^{i-1} + \dots + m_{i-2} \times 2 + m_{i-1}$. The operation $Push_M(Q, i)$ pushes the least significant i bits of an integer Q to the stack.

2.2 Related works

2.2.1 Near-optimal RDH Method using Arithmetic Coding [27]

Lin et al. [27] found that the optimal transition probability $P_{Y|X}(y|x)$ in (1) can be derived from P_X and P_Y . Given

the host PMF P_X by the host sequence, a backward and forward iterative (BFI) algorithm (see [27] for more details) was proposed to estimate the optimal stego cumulative PMF P_{CY} from the host cumulative PMF P_{CX} . In general, the BFI algorithm is an iterative algorithm to approach the unique solution of $P_{CY} = \{P_{CY}(k) | k \in \Sigma_{B+1}\}$. Specifically, it first initializes a set $E = \{e_k = k/B, k \in \Sigma_{B+1}\}$, where the variable e_k stores each temporary value of $P_{CY}(k)$. Second, it updates each e_k^{new} in alternating forward and backward manner, which achieves faster convergence than updating in a one-way manner. Finally, the algorithm converges to the optimal stego cumulative PMF P_{CY} . With the host and stego PMFs, the RDH algorithm, abbreviated as AC-RDH, was proposed based on arithmetic coding. Precisely, the embedder maintains two values $(l^{(i)}, u^{(i)}) \subseteq [0, 1]$, $i = 1, 2, \dots, N$ to interpret the joint information of host signal x_i and the temporal information produced at the last step. At the i -th step, the embedder transforms the vector $(l^{(i)}, u^{(i)})$ and the next host signal x_{i+1} to the stego signal y_i and the next vector $(l^{(i+1)}, u^{(i+1)})$, expressed as $(y_i, l^{(i+1)}, u^{(i+1)}) = \text{Encode}(x_{i+1}, l^{(i)}, u^{(i)})$.

In the theoretical perspective, maintaining $l^{(i)}$ and $u^{(i)}$ requires infinite-precision real numbers, which is computationally unfeasible, and thus the authors adopt $b^{(i)}$ -bit integer arithmetic to implement the $l^{(i)}$ and $u^{(i)}$. To ensure that the coding interval is always large enough, $b^{(i)}$ shall be increased during the encoding process. However, modern CPUs are usually designed for 32 or 64-bit arithmetic, and they cannot support extremely high-precision calculations directly. When $b^{(i)} > 64$ higher than the precision supported by CPUs, the upper and lower bounds coincide $l^{(i)} = u^{(i)}$, and this causes AC-RDH cannot encode any more bits.

2.2.2 A Variant of ANS Coding [35]

ANS is a new entropy encoding and has been integrated into commercial compressors. The pure form of ANS coding encodes an input sequence of symbols into a single natural number (also called state). Before the encoding, ANS holds a start state s . On a high level, the ANS encoder always maintains a state that can be modified by the encoding function $s' = C(x, s) \in \mathbb{N}$. The function $C(x, s)$ receives a symbol x and the current state s to produce a new state s' . Correspondingly, the decoding function decodes the state s' to a symbol x and the previous state s . It is noted that the order of symbols processed by the ANS encoder is opposite to that reconstructed by the ANS decoder.

Recently [35], a variant of ANS coding (abbreviated ANS-Variant coding) has been proposed to provide high compression performance. Consider an input sequence $X = (x_1, x_2, \dots, x_N)$, where $N = 2^n$ for a positive integer n , for each symbol x , the ANS-Variant coding first outputs an n -bit value, and this value can be used to decode the symbol. Then, the encoder performs the encoding function $s' = C(x, s)$. After a symbol is encoded, we have $s' \leq s$. In addition, s is renormalized to stay within a fixed interval I by reading bits from the output stream to enlarge it if s gets too small in the encoder or writing some bits of the state into the output stream to reduce it if s gets too large in the decoder. Let I_x denote the interval corresponding to the symbol x , which ensures that the new state after encoding symbol x is within the interval I . The encoding

details are given below. To encode a symbol x , if the current state s before encoding is within I_x , the encoder directly outputs an n -bit value and produces a new state by $C(x, s)$. Otherwise, the encoder first reads n bits and appends them to the least significant n bits of state s to increase it, then the encoder outputs an n -bit value and performs $C(x, s)$ with the increased s . We continue the above steps for the next symbol until all symbols are encoded.

In decoding, given a start state for decoding (which is exactly the final state in encoding) and the output stream, the decoder first reads an n -bit value and decodes it to a symbol x . Then, we update the state s to enlarge it. If the enlarged state s exceeds the upper bound chosen during the encoding, the decoder writes the least significant n bits of state s to the output stream to make s smaller. We repeat the above steps, and the repetition stops when all symbols are decoded.

3 PROPOSED STATIC METHOD

In this section, we first propose a coding method that employs ANS-Variant coding to achieve data embedding. The primary advantage of selecting it over the traditional ANS code for RDH lies in its ability to reduce computational costs, thereby enhancing the overall performance of the system. Then, the corresponding data extraction procedure is also given. This section considers the case that the embedder and extractor know the host PMF P_{CX} , thus called it static method. The proposed static RDH method includes static data embedding and static data extraction. Among them, the static embedding method mainly aims at constructing a coding scheme that can approach the theoretical rate-distortion bound, and the additional overhead in the static extraction is independent of the volume of both the host media and the secret data, but only related to the alphabet size.

3.1 Static Data Embedding

Based on the BFI algorithm [27], we first find out the optimal cumulative PMF P_{CY} for the host sequence X . Then, this subsection presents an entropy coding method to embed the message W into the host sequence $X = (x_1, x_2, \dots, x_N)$ via the given cumulative PMFs P_{CX} and P_{CY} . The proposed algorithm adopts ANS-Variant coding with fixed precision as the coding framework for RDH. The details of each step are as follows.

3.1.1 Estimating the Stego P_{CY}

In this step, the BFI algorithm [27] is used to find out the optimal cumulative PMF P_{CY} of the stego sequence Y for a given host cumulative PMF P_{CX} . Precisely, we first get knowledge of the host statistics $F_X = \{f_X(x) | x \in \Sigma_B\}$ and $CF_X = \{cf_X(k) = \sum_{z=-1}^{k-1} f_X(z) | k \in \Sigma_{B+1}\}$, where $f_X(x)$ is the frequency of symbol x in the host sequence $X = (x_1, x_2, \dots, x_N)$. Then, we calculate the cumulative PMF $P_{CX} = \{P_{CX}(k) = \sum_{z=-1}^{k-1} P_X(z) | k \in \Sigma_{B+1}\}$, where $P_X(z) = \frac{f_X(z)}{N}$. Next, we adopt the BFI algorithm to generate a stego distribution P_{CY} that is more uniform than the host distribution P_{CX} . Please see Algorithm 1 of [27] for more details.

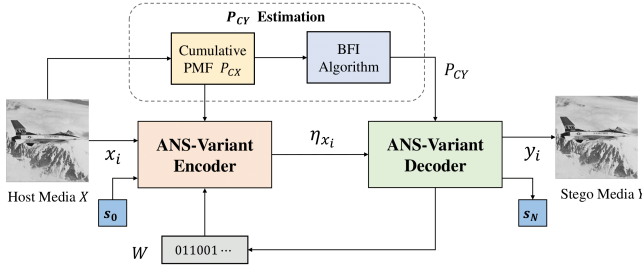


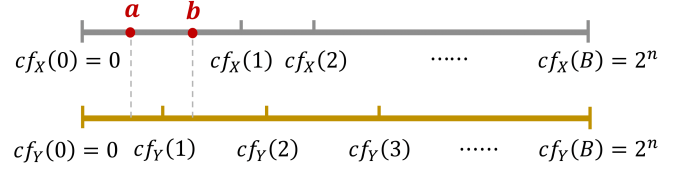
Fig. 2: Flowchart of the proposed static embedding scheme.

3.1.2 ANS-Variant Coding-based Embedding Algorithm

After obtaining the optimal stego cumulative PMF P_{CY} , we can embed the message W into the host sequence $X = (x_1, x_2, \dots, x_N)$ by utilizing the cumulative PMFs P_{CX} and P_{CY} . This relies on the fact that the entropy of distribution P_{CY} is higher than that of P_{CX} , and thus we can embed data in the stego sequence. Figure 2 shows the flowchart of the proposed static embedding scheme. We process the host signals forwards, i.e., from first to last, in a signal-by-signal manner and convert each host signal x_i to stego signal y_i . The proposed embedding algorithm combines an ANS-Variant encoder and an ANS-Variant decoder, where the encoder receives the host PMF P_{CX} , a start state s_0 and encodes each host signal x_i . The output η_{x_i} of the ANS-Variant encoder and the stego PMF P_{CY} are transmitted to the ANS-Variant decoder to produce the corresponding stego signal y_i . Precisely, the procedure of our embedding module consists of the following three steps.

- (1) **Employing State Value as an Indicator for Embedding:** As the probability of host signal x_i and the stego signal y_i which it encodes into is different, the self-information between these two signals are also different. The gap between the self-information of the host signal and the stego signal it encodes into creates a chance to "hide" the secret information. However, as the smallest unit of secret data is 1 bit, but sometimes this gap may be less than 1 bit, this leads to embedding impossible. Therefore, in this paper, we employ the size of the encoding state as an indicator to show the size of the accumulated gap. When the accumulated gap is large enough, we perform a practical embedding together. For example, if the accumulated gap is more than v bits, we can embed v bits.

Let s_{i-1} be the state before processing the i -th host signal. We forward process the host signals, i starts from 1. Given the host statistics $F_X = \{f_X(x) | x \in \Sigma_B\}$, $CF_X = \{cf_X(k) = \sum_{z=1}^{k-1} f_X(z) | k \in \Sigma_{B+1}\}$, and a start state s_0 , where $\sum_{x \in \Sigma_B} f_X(x) = 2^n$ for $n \in \mathbb{N}^+$ and it is a standard assumption for ANS coding to simplify arithmetic operations, we use the size of the state as an indicator to decide whether to embed secret data or not. Precisely, let $I := [2^{T-v \times n}, 2^T)$ and $I_{x_i} := [f_X(x_i) \times 2^{T-v \times n}, 2^T)$, where $T, v \in \mathbb{N}^+$ as long as $T - v \times n \geq 0$, we discuss the state s_{i-1} before encoding the host signal x_i in the following two cases. In the first, if the state s_{i-1} is less than the lower bound of the interval I_{x_i} , i.e., $s_{i-1} < f_X(x_i) \times 2^{T-v \times n}$, we read $v \times n$ bits from the message W to increase s_{i-1} . For

Fig. 3: Diagram showing relations of CF_X and CF_Y with intervals.

example, when the decimal representation of the $v \times n$ bits read is D , we have

$$s_{i-1} \leftarrow (s_{i-1} \ll v \times n) + D, \quad (4)$$

where $\ll$ denotes the bitwise left-shift operation. In the second, if the state s_{i-1} is exactly within the interval I_{x_i} , no change on the state is needed.

- (2) **Performing the ANS-Variant Encoding:** After the state s_{i-1} is adjusted to be within the interval I_{x_i} , we then use an ANS-Variant encoder to encode the host signal x_i . Let $Rem(i, j)$ return the remainder after the division of i by j , where i is the divided and j is the divisor. Precisely, we first hold an n -bit temporal information

$$\eta_{x_i} = cf_X(x_i) + Rem(s_{i-1}, f_X(x_i)), \quad (5)$$

and then update the state s_{i-1} by

$$s_i \leftarrow C(x_i, s_{i-1}) = \lfloor s_{i-1} / f_X(x_i) \rfloor, \quad (6)$$

where $\lfloor \cdot \rfloor$ returns the greatest integer less than or equal to the input number. It is noted that the above n -bit temporary information η_{x_i} in (5) will be used in the later Step (3) to yield the stego signal y_i at the corresponding position.

- (3) **Generating Stego Signal at the Corresponding Position:** In Section 3.1.1, we estimated the stego cumulative PMF $P_{CY} = \{P_{CY}(k) | k \in \Sigma_{B+1}\}$ for a given host PMF P_{CX} . After the stego distribution is obtained, we can calculate the stego statistics $F_Y = \{f_Y(k) | k \in \Sigma_B\}$ and $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$ by

$$cf_Y(k) = \lfloor P_{CY}(k) \times N \rfloor, \quad f_Y(k) = cf_Y(k+1) - cf_Y(k). \quad (7)$$

Next, we utilize the temporary information η_{x_i} maintained in (5) and the above stego statistics to decode the stego signal y_i . Precisely, let $\tilde{c}_Y(i) \triangleq y$ if $cf_Y(y) \leq i < cf_Y(y+1)$ for $i \in [0, 2^n)$. We transmit the temporary information η_{x_i} to the ANS-Variant decoder to obtain stego signal y_i , i.e., $y_i \leftarrow \tilde{c}_Y(\eta_{x_i})$. For example, Figure 3 shows the graphical representation of the cumulative frequency distribution $CF_X = \{cf_X(k) | k \in \Sigma_{B+1}\}$ and $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$ with intervals, where the stego sequence Y is more uniformly distributed than X . In Figure 3, if the host signal x_i is 0, we have $cf_X(0) = 0 \leq \eta_{x_i} < cf_X(1)$, so its stego signal y_i has the following two cases. In the first, if $cf_Y(0) = 0 \leq \eta_{x_i} < cf_Y(1)$, e.g., $\eta_{x_i} = a$, we have $y_i = 0$. In the second, if $cf_Y(1) \leq \eta_{x_i} < cf_X(1)$, e.g., $\eta_{x_i} = b$, we have $y_i = 1$. After the stego signal y_i is produced, we then update the state s_i by

$$s_i \leftarrow f_Y(y_i) \times s_i + r_{y_i}, \quad (8)$$

Algorithm 1: Proposed static embedding algorithm

Input: A host sequence $X = (x_1, x_2, \dots, x_N)$, a message W , the alphabet size B , the cumulative PMF P_{CX} .

Output: A stego sequence $Y = (y_1, y_2, \dots, y_N)$, the final state s_N , the $v \times n$ bits pushed in step N (if that occurs).

- 1 Get the host statistics $F_X = \{f_X(k) | k \in \Sigma_B\}$ and $CF_X = \{cf_X(k) | k \in \Sigma_{B+1}\}$ from the P_{CX} ;
- 2 Use the BFI algorithm to estimate the stego P_{CY} ;
- 3 Get the stego statistics $F_Y = \{f_Y(k) | k \in \Sigma_B\}$ and $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$ from the P_{CY} ;
- 4 Set a start state $s_0 \leftarrow 1$;
- 5 **for** $i = 1$ **to** N **do**
- 6 **if** $s_{i-1} < f_X(x_i) \times 2^{T-v \times n}$ **then**
- 7 $D \leftarrow Pop_W(v \times n)$;
- 8 $s_{i-1} \leftarrow (s_{i-1} \ll v \times n) + D$;
- 9 $\eta_{x_i} = cf_X(x_i) + Rem(s_{i-1}, f_X(x_i))$;
- 10 $s_i \leftarrow \lfloor s_{i-1} / f_X(x_i) \rfloor$;
- 11 $y_i \leftarrow \tilde{c}_Y(\eta_{x_i})$;
- 12 $s_i \leftarrow f_Y(y_i) \times s_i + \eta_{x_i} - cf_Y(y_i)$;
- 13 **if** $s_i \geq 2^T$ **then**
- 14 call $Push_W(s_i, v \times n)$;
- 15 $s_i \leftarrow s_i \gg v \times n$;

where $r_{y_i} = \eta_{x_i} - cf_Y(y_i)$. The updated state s_i in (8) has the following two cases. In the first, if the value of state s_i exceeds the upper bound 2^T chosen in the encoding, we write the least significant $v \times n$ bits of s_i to the message W , i.e., $Push_W(s_i, v \times n)$, and make s_i smaller by

$$s_i \leftarrow s_i \gg v \times n, \quad (9)$$

where $\gg$ denotes the bitwise right-shift operation. In the second, if the updated state s_i is exactly within the interval I , no further changes to the state are needed.

We repeat the above three steps on all host signals. Algorithm 1 presents the details. In Algorithm 1, Line 1 gets the host statistics F_X and CF_X . Lines 2–3 first apply the BFI algorithm to estimate the optimal P_{CY} and then obtain the stego statistics F_Y and CF_Y from the P_{CY} . Line 4 sets a start state for ANS-Variant coding. Lines 6–10 perform the encoding of ANS-Variant coding. In particular, Lines 7–8 read bits from the message W to enlarge the state if it is too small. Lines 11–15 perform the decoding of ANS-Variant coding to yield the corresponding stego signal y_i . Finally, we get a stego sequence $Y = (y_1, y_2, \dots, y_N)$ and a final state s_N . Note that the final state s_N needs to be transmitted along with the stego sequence Y such that the extractor knows what state to start with. In addition, if one wants to recover the message from the stego sequence (see Section 3.2), we have to additionally store the $v \times n$ bits written to the message W in the N -th step (if that occurs). In our experiments, we select the parameters $v = 1$ and $n = 16$, so this overhead is negligible.

3.2 Static Data Extraction

In this subsection, we show how to losslessly reconstruct the host sequence X and the embedded message W from

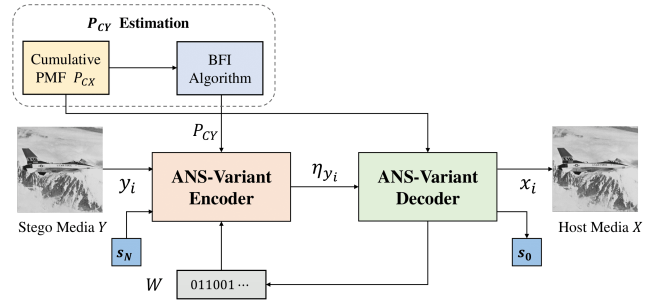


Fig. 4: Flowchart of the proposed static extraction scheme.

the received stego sequence Y when the host cumulative PMF P_{CX} is known (or both P_{CX} and P_{CY} are known but this costs more space overhead). The data extraction is exactly the dual process of data embedding. Figure 4 shows the flowchart of the proposed static extraction scheme. We process the stego signals backward, i.e., from last to first, in a signal-by-signal manner, and revert each stego signal y_i to host signal x_i . Specifically, we first apply the BFI algorithm on P_{CX} to estimate the stego P_{CY} and perform the ANS-Variant encoding on stego signal y_i based on the stego statistics derived from the P_{CY} . Then, the output of the previous ANS-Variant encoder and the host statistics are transferred to an ANS-Variant decoder to decode the host signal x_i . Similarly, during the decoding of host signal x_i , we employ the size of the state as an indicator for extracting secret data.

Given a stego sequence $Y = (y_1, y_2, \dots, y_N)$, a start state s_N , the host cumulative PMF $P_{CX} = \{P_{CX}(k) | k \in \Sigma_{B+1}\}$, and the $v \times n$ bits pushed in step N during the embedding (if that occurs), we first fill an empty message W with these $v \times n$ bits. Then, the details of our data extraction are as follows.

(1) **Performing the ANS-Variant Encoding on Stego Signal:** Let s_i^* be the state before processing the i -th stego signal. We backward process the stego signals, i starts from N . First, we apply BFI algorithm on P_{CX} to estimate the stego $P_{CY} = \{P_{CY}(k) | k \in \Sigma_{B+1}\}$. Based on the P_{CY} , we have stego statistics $F_Y = \{f_Y(k) | k \in \Sigma_B\}$ and $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$, where $cf_Y(k)$ and $f_Y(k)$ are calculated by (7). Then, for each stego signal y_i , we perform the ANS-Variant encoding based on the obtained stego statistics F_Y and CF_Y . Concretely, let $I := [2^{T-v \times n}, 2^T)$ and $I_{y_i} := [f_Y(y_i) \times 2^{T-v \times n}, 2^T)$. In the encoding of signal y_i , we have the following two cases. In the first, if $s_i^* < f_Y(y_i) \times 2^{T-v \times n}$, we read $v \times n$ bits from the message W to increase s_i^* by (4). Then, we hold an n -bit temporal information

$$\eta_{y_i} = cf_Y(y_i) + Rem(s_i^*, f_Y(y_i)), \quad (10)$$

and update the state s_i^* by

$$s_i^* \leftarrow C(y_i, s_i^*) = \lfloor s_i^* / f_Y(y_i) \rfloor. \quad (11)$$

In the second, if the current state s_i^* is within I_{y_i} , we directly maintain an n -bit temporal information η_{y_i} by (10) and produce a new state s_i^* by (11). It is noted that the n -bit temporal information η_{y_i} produced here will be used in the later Step (2) to decode the corresponding host signal x_i .

- (2) **Reconstructing Host Signal at the Corresponding Position:** According to the given host cumulative PMF P_{CX} , we can get the host statistics $F_X = \{f_X(k) | k \in \Sigma_B\}$ and $CF_X = \{cf_X(k) | k \in \Sigma_{B+1}\}$ via

$$cf_X(k) = \lfloor P_{CX}(k) \times N \rfloor, \quad f_X(k) = cf_X(k+1) - cf_X(k). \quad (12)$$

Next, we utilize the temporary information η_{y_i} maintained in the (10) and the above host statistics to decode the host signal x_i . Let $\tilde{c}_X(i) \triangleq x$ if $cf_X(x) \leq i < cf_X(x+1)$ for $i \in [0, 2^n]$. Precisely, we transmit the temporary information η_{y_i} to the ANS-Variant decoder to decode x_i , i.e., $x_i \leftarrow \tilde{c}_X(\eta_{y_i})$. Then, we update the state s_i^* by

$$s_{i-1}^* \leftarrow f_X(x_i) \times s_i^* + r_{x_i}, \quad (13)$$

where $r_{x_i} = \eta_{y_i} - cf_X(x_i)$. As a result, we can obtain the host signal x_i at the corresponding position.

- (3) **Employing State Value as an Indicator for Extracting Secret Data:** As mentioned, the secret data is hiding within the gap between the coding rate of the stego signal and the host signal. Hence, the most important problem in extracting the secret data is to identify where the gap is. To address this issue, we take the state in (13) as the indicator for extracting secret data. Note that the indicator for the i -th step in data extraction s_{N+1-i}^* is the same as the indicator for the $(N+1-i)$ -th step in data embedding s_{N+1-i} , since the operation on s_{N+1-i}^* is the reverse operation on s_{N+1-i} , so we can figure out whether a gap occurs by the size of the updated state in (13). Specifically, if the value of state s_{i-1}^* in (13) exceeds the upper bound 2^T chosen in the data embedding, the gap occurs. In this case, we write the least significant $v \times n$ bits of state s_{i-1}^* to the message W as the recovered secret data, and make s_{i-1}^* smaller by (9). Otherwise, no further changes to the state are needed.

We repeat the above steps on all stego signals. Algorithm 2 gives the details. In Algorithm 2, Line 2 uses the BFI algorithm to estimate the optimal P_{CY} . Lines 3–4 derive the host and stego statistics from the known P_{CX} and P_{CY} , respectively. Lines 6–10 perform the encoding of ANS-Variant coding on the stego signal and hold temporary information η_{y_i} . Lines 11–15 utilize the η_{y_i} maintained in the previous step to decode the host signal x_i . In particular, Lines 13–15 write $v \times n$ bits to the message W when the state exceeds the upper bound 2^T . Finally, we can losslessly reconstruct the host sequence $X = (x_1, x_2, \dots, x_N)$ and the embedded message W .

4 PROPOSED DYNAMIC METHOD

From Figure 4, one can see that the proposed static method requires a host PMF P_{CX} in advance in the extraction procedure, which brings an additional transmission overhead. Therefore, in this section, we present a dynamic method without the need to transfer the host PMF P_{CX} . In general, the embedding and extraction procedures of the proposed dynamic method are based on an assumed PMF, which can be pre-shared between the embedder and the extractor through a secure and reliable channel before the communication begins. Moreover, since the assumed PMF is not accurate, the proposed algorithm adjusts this PMF during

Algorithm 2: Proposed static extraction algorithm

Input: A stego sequence $Y = (y_1, y_2, \dots, y_N)$, a start state s_N , the $v \times n$ bits pushed in step N during the embedding (if that occurs), the alphabet size B , the cumulative PMF P_{CX} .
Output: A host sequence $X = (x_1, x_2, \dots, x_N)$, the embedded message W .

```

1 Let  $s_N^* \leftarrow s_N$ ;
2 Use the BFI algorithm to estimate the optimal  $P_{CY}$ ;
3 Get the host statistics  $F_X = \{f_X(k) | k \in \Sigma_B\}$  and
   $CF_X = \{cf_X(k) | k \in \Sigma_{B+1}\}$  from the  $P_{CX}$ ;
4 Get the stego statistics  $F_Y = \{f_Y(k) | k \in \Sigma_B\}$  and
   $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$  from the  $P_{CY}$ ;
5 for  $i = N$  to 1 do
6   if  $s_i^* < f_Y(y_i) \times 2^{T-v \times n}$  then
7      $D \leftarrow Pop_W(v \times n)$ ;
8      $s_i^* \leftarrow (s_i^* \ll v \times n) + D$ ;
9    $\eta_{y_i} = cf_Y(y_i) + Rem(s_i^*, f_Y(y_i))$ ;
10   $s_i^* \leftarrow \lfloor s_i^* / f_Y(y_i) \rfloor$ ;
11   $x_i \leftarrow \tilde{c}_X(\eta_{y_i})$ ;
12   $s_{i-1}^* \leftarrow f_X(x_i) \times s_i^* + \eta_{y_i} - cf_X(x_i)$ ;
13  if  $s_{i-1}^* \geq 2^T$  then
14    call  $Push_W(s_{i-1}^*, v \times n)$ ;
15     $s_{i-1}^* \leftarrow s_{i-1}^* \gg v \times n$ ;

```

the coding process instead of using this untrustworthy PMF to encode the host signal directly. Specifically, we first agree on a given distribution as the assumed PMF, and this PMF is known for both the data embedder and the extractor.

Meanwhile, in order to recover the host sequence and secret data without errors, the PMFs used by the embedder and extractor must be completely identical (synchronized) when processing the same signal. That is, the adjustment is required to be a reversible process. Therefore, we design a reversible dynamic probability calculator that synchronizes the PMF in the case where the symbols recovered by the decoder are in the opposite order of the symbols received by the encoder.

Our dynamic method involves the following two parts. First, we describe the dynamic data embedding procedure. Second, the details of dynamic data extraction are given.

4.1 Dynamic Data Embedding

In general, the proposed dynamic embedding framework is similar to that of the static embedding scheme, but it uses a varying PMF instead of a constant PMF. That is, our dynamic method changes the PMF after each host signal is processed. Figure 5 shows the flowchart of the proposed dynamic embedding scheme. Specifically, for a given assumed frequency distribution F_X , the proposed dynamic embedding scheme consists of the following two steps. First, we count the frequency distribution of the host sequence and accumulate it to F_X . Second, based on the newest F_X , we process the host signals forwards, i.e., from first to last in a signal-by-signal manner, and adjust the F_X after each host signal is encountered. The adjusted F_X is used for subsequent encoding of this signal.

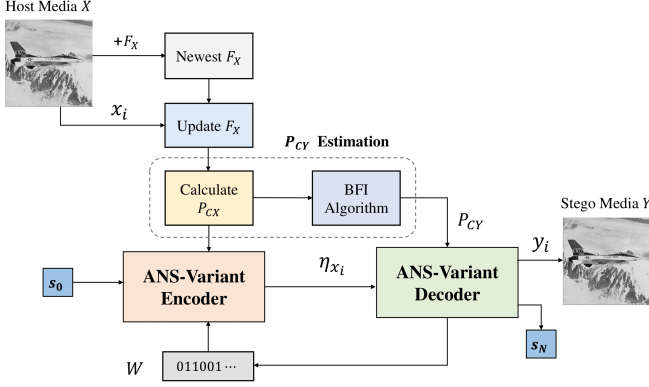


Fig. 5: Flowchart of the proposed dynamic embedding scheme.

Given a host sequence $X = (x_1, x_2, \dots, x_N)$ and a assumed frequency distribution $F_X = \{f_X(x) | x \in \Sigma_B\}$, where $f_X(x)$ is the frequency of host signal x , the details are as follows:

- (1) **Get Statistics:** We first pass the host sequence X once to count its frequency distribution. Precisely, when a host signal x_i is encountered, we update the frequency corresponding to the signal x_i via

$$f_X(x_i) \leftarrow f_X(x_i) + 1. \quad (14)$$

We continue with the next host signal and update its frequency in the same manner. Finally, we can get the newest frequency distribution F_X .

- (2) **Forward Pass and Embed:** After the frequency distribution of the host sequence is obtained, we perform the following data embedding based on the newest statistics $F_X = \{f_X(x) | x \in \Sigma_B\}$. For each host signal x_i encountered, we first update its frequency by

$$f_X(x_i) \leftarrow f_X(x_i) - 1. \quad (15)$$

Let $F_X^i = \{f_X(x) | x \in \Sigma_B\}$ denote the frequency distribution at the i -th step. From (15), one can see that the distribution is changing. Then, we have the host statistics

$$CF_X = \left\{ cf_X(k) = \sum_{z=-1}^{k-1} f_X(z) | k \in \Sigma_{B+1} \right\} \quad (16)$$

and $P_{CX} = \{cf_X(k)/cf_X(B) | k \in \Sigma_{B+1}\}$ from $F_X^i = \{f_X(x) | x \in \Sigma_B\}$. Next, we apply the BFI algorithm on the P_{CX} to estimate the optimal stego P_{CY} of the stego sequence Y . Once the $P_{CY} = \{P_{CY}(k) | k \in \Sigma_{B+1}\}$ is known, we can also obtain the stego statistics $F_Y = \{f_Y(k) | k \in \Sigma_B\}$ and $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$, where $cf_Y(k)$ and $f_Y(k)$ are calculated by (7). When the host and stego statistics (i.e., F_X , CF_X , F_Y and CF_Y) are all available, we then implement data embedding using similar steps in static data embedding (see Lines 6–15 in Algorithm 1 for details).

We repeat the above steps on all host signals. Algorithm 3 gives the details. In Algorithm 3, Lines 1–3 initialize the frequency distribution F_X to all ones and count the host's frequencies. Line 6 updates the frequency corresponding to

Algorithm 3: Proposed dynamic embedding algorithm

Input: A host sequence $X = (x_1, x_2, \dots, x_N)$, a message W , the alphabet size B .

Output: A stego sequence $Y = (y_1, y_2, \dots, y_N)$, the final state s_N , the $v \times n$ bits pushed in step N (if that occurs).

```

1 Initialize  $F_X = \{f_X(x) = 1 | x \in \Sigma_B\}$ ;
2 for  $i = N$  to 1 do
3    $f_X(x_i) \leftarrow f_X(x_i) + 1$ ;
4 Set a start state  $s_0 \leftarrow 1$ ;
5 for  $i = 1$  to  $N$  do
6    $f_X(x_i) \leftarrow f_X(x_i) - 1$ ;
7   Get the host statistic  $CF_X = \{cf_X(k) | k \in \Sigma_{B+1}\}$ 
   and  $P_{CX}$  from the  $F_X^i = \{f_X(x) | x \in \Sigma_B\}$ ;
8   Use the BFI algorithm to estimate the  $P_{CY}$ ;
9   Get the stego statistics  $F_Y = \{f_Y(k) | k \in \Sigma_B\}$  and
    $CF_Y = \{cf_Y(k) | k \in \Sigma_{B+1}\}$  from the  $P_{CY}$ ;
10  if  $s_{i-1} < f_X(x_i) \times 2^{T-v \times n}$  then
11     $D \leftarrow PopW(v \times n)$ ;
12     $s_{i-1} \leftarrow (s_{i-1} \ll v \times n) + D$ ;
13   $\eta_{x_i} = cf_X(x_i) + Rem(s_{i-1}, f_X(x_i))$ ;
14   $s_i \leftarrow \lfloor s_{i-1} / f_X(x_i) \rfloor$ ;
15   $y_i \leftarrow \tilde{c}_Y(\eta_{x_i})$ ;
16   $s_i \leftarrow f_Y(y_i) \times s_i + \eta_{x_i} - cf_Y(y_i)$ ;
17  if  $s_i \geq 2^T$  then
18    call  $PushW(s_i, v \times n)$ ;
19     $s_i \leftarrow s_i \gg v \times n$ ;

```

the signal x_i . Lines 7–8 first calculate the host statistics CF_X and P_{CX} , and then use the BFI algorithm to estimate the optimal P_{CY} . Line 9 obtains the stego statistics F_Y and CF_Y from the P_{CY} . Lines 10–14 perform the encoding of ANS-Variant coding. In particular, Lines 11–12 read bits from the message W to enlarge the state if it is too small. Lines 15–19 perform the decoding of ANS-Variant coding to obtain the corresponding stego signal y_i . Finally, we get a stego sequence $Y = (y_1, y_2, \dots, y_N)$ and a final state s_N .

4.2 Dynamic Data Extraction

In this subsection, we show how to losslessly recover the host sequence X and the secret message W from the received stego sequence Y in the absence of host PMF P_X . The proposed dynamic data extraction procedure consists of the following two steps. First, we start with the same assumed frequency distribution and calculate the host PMF based on the current distribution. Then, we estimate the stego PMF and perform data extraction similar to static data extraction. Note that we update the distribution after each host signal is decoded.

As mentioned, it is crucial to keep the distributions received by the embedder and extractor synchronized when processing the same signal. Here, we repeat the inverse process of frequency distribution adjustment in the dynamic embedding process to achieve it. Figure 6 shows the flowchart of the proposed dynamic extraction scheme. Specifically, given a stego sequence $Y = (y_1, y_2, \dots, y_N)$,

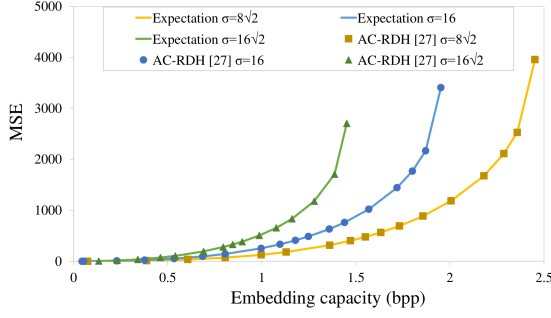


Fig. 7: Rate-distortion curves of a given normal distribution for various standard deviation σ , and rate-distortion values of AC-RDH [27].

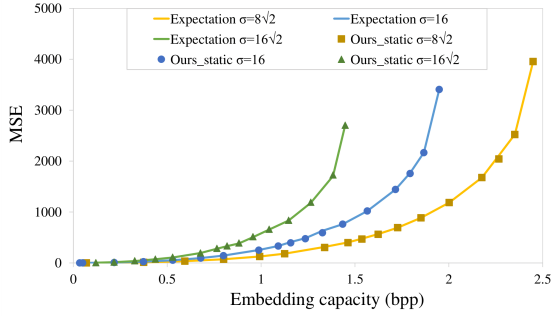


Fig. 8: Rate-distortion curves of a given normal distribution for various standard deviation σ , and rate-distortion values of the proposed static method.

- LPVO [11] utilizes pixel value orders and pixel positions for prediction and selects mappings based on image content. It is well known for high stego image quality.
- MHM [18] expands multiple pairs of bins in each PEH and develops an optimal modification, resulting in high embedding performance.
- IT-RDH [36] exploits image textures for data embedding, and the secret data is preferentially embedded in the position of smooth pixels for better stego image quality.

Our proposed methods are implemented in Visual C++ code with OpenCV image development kits. We use 32 bits to represent the state and choose parameters $T = 16$, $n = 16$ and $v = 1$. Throughout the simulation, the secret message W is generated by multiple calls to the operation $RAND() * 65536$, which returns a pseudo-random number between 0 (inclusive) and 65536 (exclusive). The host media used for RDH include standard distributions and images. First, we show the experimental results on the i.i.d. sequences drawn from discrete normal distributions. Second, all experiments are conducted on the standard gray-scale images of size 512×512 .

5.1 Experiments for i.i.d. Sequences

Similar to AC-RDH [27], to evaluate whether the performance of the proposed method is close to the expected rate-distortion bound, we tested the embedding performance

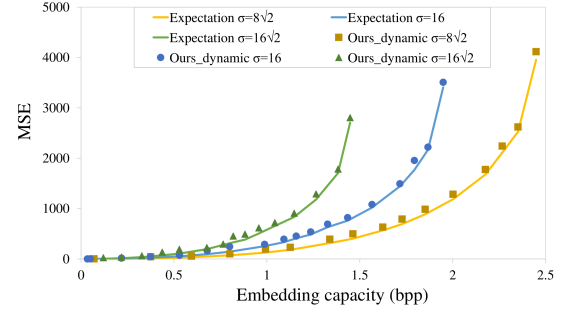


Fig. 9: Rate-distortion curves of a given normal distribution for various standard deviation σ , and rate-distortion values of the proposed dynamic method.

on the i.i.d. sequence drawn from a discrete normal distribution. We implemented AC-RDH, the proposed static method (Ours_static) and the proposed dynamic method (Ours_dynamic) to embed the message in 65536 host signals. The host signals are 8-bit grayscale with $B = 256$, and the mean of the normal distribution is 127.5. We adopt two common metrics, embedding capacity (bpp) and MSE, to evaluate the embedding performance.

Figures 7–9 show the rate-distortion curves for the P_{CX} with standard deviations σ , where the yellow line, blue line and green line, respectively, depict the expected rate-distortion (abbreviated as Expectation, which refers to the minimum distortion for the host sequence to achieve a given embedding capacity) curves for $\sigma = 8\sqrt{2}$, 16 and $16\sqrt{2}$, and the squares, circles and triangles, respectively, mark the rate-distortion values of the implementations. From Figures 7–8, one can see that the proposed static method performs similarly to AC-RDH, and both are very close to the expected rate-distortion bound. For example, when $\sigma = 8\sqrt{2}$ and the embedding rate is about equal to 2.0 bpp, the MSEs of the expectation, AC-RDH and the proposed static method are 1185.10, 1185.28 and 1185.24, respectively.

Figure 9 demonstrates the performance of the proposed dynamic method. It shows that the rate-distortion values in the dynamic version have a small distance from the expected rate-distortion bound. For example, when $\sigma = 8\sqrt{2}$ and the embedding rate is about equal to 2.0 bpp, the MSEs of the expectation and the proposed dynamic method are 1185.10 and 1265.05, respectively. This is due to the fact that we use a varying PMF instead of the true PMF. Note that our dynamic proposal does not require the host PMF in advance, thus saving the additional overhead.

All in all, experimental results on discrete normally distributed signals show that the performance of the proposed static method is comparable to that of AC-RDH, and both are close to the expected rate-distortion bounds. Moreover, the later simulation results (see Section 5.2) on gray-scale images show that the proposed static method can provide higher PSNR values and larger embedding capacity than AC-RDH. In addition, the proposed dynamic method saves the space overhead that is unavoidable in AC-RDH and obtains satisfactory embedding capacity at the expense of slightly higher distortion.

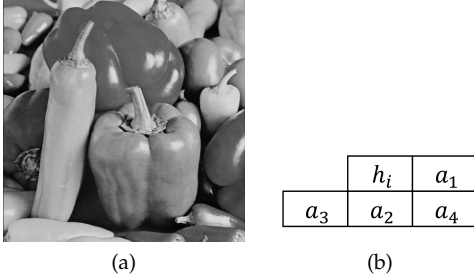


Fig. 10: (a). Host image Peppers. (b). Four neighboring pixels adopted in the predictor.

5.2 Experiments for Gray-Scale Images

In this subsection, the embedding performance of the proposed static/dynamic method on gray-scale images is compared with four state-of-the-art works, including LPVO [11], MHM [18], AC-RDH [27] and IT-RDH [36].

5.2.1 Image Preprocessing

To reduce the correlation of the neighboring image pixels, we preprocess the host image with predictive coding. Note that the performance of the RDH scheme would be improved if someone chooses an advanced predictor. For the sake of simplicity, we select the simplest predictor as follows. The predictive coding processes the host pixels from left to right and from top to bottom. Figure 10 shows a 512×512 host image Peppers and depicts the four neighboring pixels used for predicting the host pixel h_i . The predicted value $\hat{h}_i$ is defined as

$$\hat{h}_i = \frac{3}{8} \times a_1 + \frac{3}{8} \times a_2 + \frac{1}{8} \times a_3 + \frac{1}{8} \times a_4. \quad (20)$$

For the pixels at the leftmost column, e.g., a_3 , we pick the nearby right pixel a_2 as the predicted value. For the pixels at the rightmost column, e.g., a_1 , we pick the nearby lower pixel a_4 as the predicted value. For the pixels at the bottom row, e.g., a_2 , we pick the nearby right pixel a_4 as the predicted value. We omit the bottom-right corner pixel and default it to 128.

In order to avoid the pixel overflow/underflow problem, we suggest developing a two-dimensional lookup table K of size 256×256 . The row index i of the table denotes the predicted pixel, and the column index j denotes the true host pixel, where $0 \leq i, j < 256$. The entry $K[i][j]$ at the i -th row and the j -th column represents the number of host pixel j predicted as predicted pixel i . The details of constructing this table are as follows. First, we initialize each row i of the two-dimensional lookup table K as a discrete uniform distribution. Then, for each host pixel h_i , we use the predictor to obtain its predicted value $\hat{h}_i$, and update the corresponding entry by $K[\hat{h}_i][h_i] \leftarrow K[\hat{h}_i][h_i] + 1$. If the predictor works accurately enough, i.e., $\hat{h}_i \approx h_i$, one can see that the larger values in the table K are clustered on the diagonal. In other words, each row nearly exhibits a sharp distribution, which is desired for data hiding. During the data embedding process, for any host pixel h_i , we first perform predictive coding to obtain the predicted value $\hat{h}_i$. Then, we utilize the corresponding row $K[\hat{h}_i]$ of size 256 as the host PMF. By using Lines 7–19 in Algorithm 3, we can

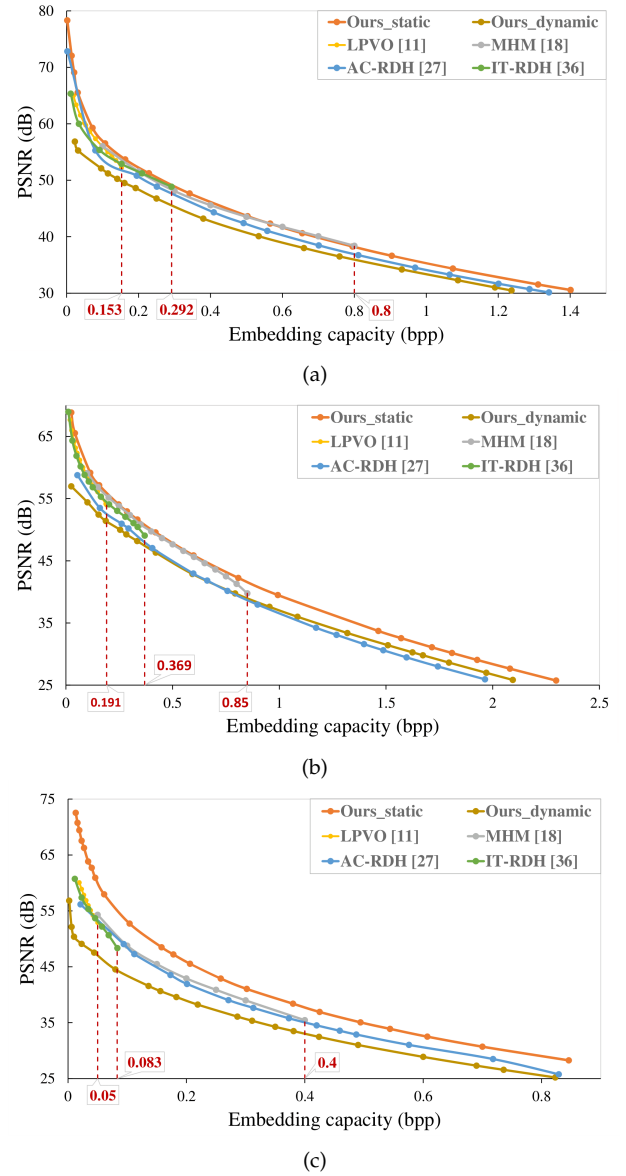


Fig. 11: Comparisons in terms of capacity-distortion performance: (a). Lena. (b). Airplane. (c). Baboon.

determine the corresponding stego pixel value that lies in the range of 0 to 255. Therefore, the problems of overflow and underflow are avoided.

5.2.2 Experimental Results

In this part, we give the experimental results on 512×512 sized standard gray-scale images. First, we show the embedding performance of the proposed static/dynamic method compared with LPVO [11], MHM [18], AC-RDH [27] and IT-RDH [36]. For page limitations, Figure 11 shows the embedding performance with respect to the embedding capacity versus the PSNR curve on three images including Lena, Airplane and Baboon from the USC_SIPi image database,¹ where the red vertical dashed lines mark the maximum embedding capacities that LPVO, IT-RDH, and MHM methods can provide. For the images Lena and

1. <https://sipi.usc.edu/database/>

TABLE 1: Comparison of maximal embedding capacities (in bpp)

Methods Images	LPVO [11]	MHM [18]	AC-RDH [27]	IT-RDH [36]	<i>Ours_dynamic</i>	<i>Ours_static</i>
Lena	0.164	0.8	3.429	0.293	2.993	3.571
Airplane	0.191	0.85	3.591	0.369	3.517	4.002
Baboon	0.050	0.4	1.700	0.083	1.400	1.876
Barbara	0.134	0.65	2.500	0.202	2.161	2.674
Lake	0.103	0.5	2.679	0.243	2.434	3.021
Boat	0.111	0.6	2.783	0.152	2.431	2.986
Elaine	0.103	0.5	2.981	0.136	2.529	3.138
Peppers	0.134	0.6	3.244	0.155	2.755	3.378
Bridge	0.118	0.5	2.108	0.148	3.455	4.221
Couple	0.107	0.65	2.777	0.183	2.638	3.247
Girlface	0.191	0.7	3.778	0.351	3.622	4.196
Zelda	0.149	0.8	3.787	0.243	3.284	3.840
Average	0.130	0.629	2.946	0.213	2.768	3.346



(a)



(b)



(c)



(d)

Fig. 12: (a). Host image Airplane. (b). Stego image, $R = 2.65$ bpp, PSNR = 22.68 dB. (c). Stego image, $R = 1.46$ bpp, PSNR = 33.7 dB. (d). Stego image, $R = 0.3$ bpp, PSNR = 53.0 dB.

Airplane, it shows that for the low embedding capacity, i.e., less than 0.5 bpp, the proposed static method provides slightly higher PSNR values than LPVO and IT-RDH. MHM achieves similar performance to the proposed static method. However, it is noted that LPVO, IT-RDH, and MHM can only yield relatively low embedding capacity, e.g., MHM produces 0.629 bpp on average (see Table 1 below). In contrast, the proposed method is capable of achieving 3.346 bpp on average. Moreover, a location map is required in MHM to prevent the underflow/overflow problem, which brings considerable additional storage consumption even after compression. Compared with AC-RDH, the proposed static method provides higher PSNR values under the same embedding capacity, and the improvement is increased for the image Airplane. The proposed dynamic method provides slightly lower PSNR values under the same embedding capacity. The reason is that we use a varying PMF instead of the true PMF to remove the need for transmitting the host PMF in advance. Sometimes, the proposed dynamic method outperforms AC-RDH, e.g., for the image Airplane with embedding capacities larger than 1.5 bpp. For the complex image Baboon, the proposed static method provides more gains over LPVO, MHM, AC-RDH and IT-RDH. Also, the proposed dynamic method removes the need to transfer the host PMF in advance at the cost of lower PSNR values. For simplicity, we initialize the assumed PMF as a discrete uniform distribution with interval $[1, B]$, where B is the alphabet size. It should be noted that with an improved assumed PMF, this method might behave better.

Second, we test the maximal embedding capacities (MEC) of these methods on 12 gray-scale images in the USC_SIPi database. Table 1 lists the results, where the table

entry is in bold if it is the highest, and the average MEC is given at the bottom. It can be seen that the proposed static method has the highest MEC for all test images, and it has the highest average MEC as well. In addition, LPVO, MHM and IT-RDH can only perform with quite low embedding capacities. The average MECs of these methods are 0.130 bpp, 0.629 bpp and 0.213 bpp, respectively. In contrast, AC-RDH and our proposed static/dynamic methods achieve an average of 2.946 bpp and 3.346/2.768 bpp, respectively. The proposed static method outperforms AC-RDH by 0.4 bpp on average in terms of the MEC. Also, the MEC of AC-RDH is on average 0.178 bpp higher than that of the proposed dynamic method. However, it is worth noting that compared with AC-RDH, the proposed dynamic method does not require a host PMF in advance. Let ψ denote the number of distinct pixels in the host image X . Suppose a discrete uniform distribution is chosen as the assumed PMF in the proposed dynamic method. In that case, we only need to store the parameters (i.e., mean and variance, each represented by a 32 bit single-precision floating-point type) of the normal distribution. In this case, the space overhead saved by our dynamic proposal in bits is computed by

$$F = \min(B \times \lceil \log_2 N \rceil, \psi \times (\lceil \log_2 B \rceil + \lceil \log_2 N \rceil)) - 2 \times 32, \quad (21)$$

where $\min(i, j)$ denotes the smaller of i and j , and $\lceil \cdot \rceil$ rounds up a number to the nearest integer greater than or equal to it. By (21), we can obtain the amount of overhead saved by the proposed dynamic method compared to the AC-RDH. For example, for the image Lena, it saves 4,544 bits.



Fig. 13: (a). Host image Baboon. (b). Stego image, $R = 1.22$ bpp, PSNR = 21.9 dB. (c). Stego image, $R = 0.38$ bpp, PSNR = 38.4 dB. (d). Stego image, $R = 0.04$ bpp, PSNR = 62.7 dB.

Finally, we show the visual results of the proposed static method. Figures 12–13 show the embedding capacity R (bpp) and PSNR (dB) of our static proposal on two 512×512 sized standard gray-scale images Airplane and Baboon, respectively (in alignment with the IEEE Code of Ethics, we omit the Lena image), where the larger R , the higher embedding capacity. It can be observed that when the embedded capacity decreases, the PSNR increases steadily. This is because the reduction in the amount of embedded information will inevitably cast less noise on images, thus increasing their quality. Also, Figures 12–13 coincide with the experimental results shown in Figure 11.

In summary, the simulations are conducted on discrete normal distribution and standard gray-scale images. Numerical results on discrete normally distributed signals show that the performance of the proposed static method is very close to the expected rate-distortion bound. Moreover, for gray-scale images, the proposed static method provides higher PSNR values and larger embedding capacities than some state-of-the-art methods. For example, our embedding capacity of image Lena is as high as 3.571 bpp. In addition, the proposed dynamic method removes the need to transfer the host PMF in advance and achieves satisfactory embedding capacities at the cost of a small stego quality loss.

6 CONCLUSION

In this paper, we proposed a high-capacity RDH code construction method that employs ANS coding as the underlying coding framework. The framework consists of an ANS encoder followed by an ANS decoder. To the best of our knowledge, this paper is the first to use ANS coding for RDH algorithms. Based on the proposed coding framework, two RDH methods are given. First, we present a static RDH method that can asymptotically approach the rate-distortion bound of RDH. Then, a dynamic RDH method is introduced to remove the need for transmitting the host PMF in advance. The simulation results, compared with some state-of-the-art methods, verified the effectiveness and superiority of our proposed methods.

7 ACKNOWLEDGMENTS

This work was supported in part by the National Key Research and Development Program of China under Grants 2022YFA1004902 and 2020YFA0712300, and the National Natural Science Foundation of China under Grants

62172280, 62071446, 62231022 and 12031011. The authors thank X. Li and B. Ou for sharing their source codes.

REFERENCES

- [1] G. Gao, S. Tong, Z. Xia, B. Wu, L. Xu, and Z. Zhao, "Reversible data hiding with automatic contrast enhancement for medical images," *Signal Processing*, vol. 178, pp. 1–14, 2021.
- [2] Z. Ren, H. Fang, J. Zhang, Z. Ma, R. Lin, W. Zhang, and N. Yu, "A robust database watermarking scheme that preserves statistical characteristics," *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–13, 2023.
- [3] W. Li, N. Li, J. Yan, Z. Zhang, P. Yu, and G. Long, "Secure and high-quality watermarking algorithms for relational database based on semantic," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 7, pp. 7440–7456, 2023.
- [4] D. Hu, D. Zhao, and S. Zheng, "A new robust approach for reversible database watermarking with distortion control," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 6, pp. 1024–1037, 2019.
- [5] S. Iftikhar, M. Kamran, and Z. Anwar, "Rrw—a robust and reversible watermarking technique for relational data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 4, pp. 1132–1145, 2015.
- [6] M. Shehab, E. Bertino, and A. Ghafoor, "Watermarking relational databases using optimization-based techniques," *IEEE Transactions on Knowledge and Data Engineering*, vol. 20, no. 1, pp. 116–129, 2008.
- [7] M. Kamran, S. Suhail, and M. Farooq, "A robust, distortion minimizing technique for watermarking relational databases using once-for-all usability constraints," *IEEE Transactions on Knowledge and Data Engineering*, vol. 25, no. 12, pp. 2694–2707, 2013.
- [8] M. Celik, G. Sharma, A. Tekalp, and E. Saber, "Lossless generalized-LSB data embedding," *IEEE Transactions on Image Processing*, vol. 14, no. 2, pp. 253–266, 2005.
- [9] J. Tian, "Reversible data embedding using a difference expansion," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 8, pp. 890–896, 2003.
- [10] A. Alattar, "Reversible watermark using the difference expansion of a generalized integer transform," *IEEE Transactions on Image Processing*, vol. 13, no. 8, pp. 1147–1156, 2004.
- [11] T. Zhang, X. Li, W. Qi, and Z. Guo, "Location-based PVO and adaptive pairwise modification for efficient

- reversible data hiding," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2306–2319, 2020.
- [12] W. He and Z. Cai, "Reversible data hiding based on dual pairwise prediction-error expansion," *IEEE Transactions on Image Processing*, vol. 30, pp. 5045–5055, 2021.
- [13] D. Wang, X. Zhang, C. Yu, and Z. Tang, "Reversible data hiding by using adaptive pixel value prediction and adaptive embedding bin selection," *IEEE Signal Processing Letters*, vol. 26, no. 11, pp. 1713–1717, 2019.
- [14] W.-L. Tai, C.-M. Yeh, and C.-C. Chang, "Reversible data hiding based on histogram modification of pixel differences," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 19, no. 6, pp. 906–910, 2009.
- [15] S. Kim, X. Qu, V. Sachnev, and H. J. Kim, "Skewed histogram shifting for reversible data hiding using a pair of extreme predictions," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 29, no. 11, pp. 3236–3246, 2019.
- [16] C. Zhang and B. Ou, "Reversible data hiding based on multiple adaptive two-dimensional prediction-error histograms modification," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 7, pp. 4174–4187, 2022.
- [17] A. Roy and R. S. Chakraborty, "Toward optimal prediction error expansion-based reversible image watermarking," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 8, pp. 2377–2390, 2020.
- [18] B. Ou and Y. Zhao, "High capacity reversible data hiding based on multiple histograms modification," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 8, pp. 2329–2342, 2020.
- [19] J. Wang, X. Chen, J. Ni, N. Mao, and Y. Shi, "Multiple histograms-based reversible data hiding: Framework and realization," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 8, pp. 2313–2328, 2020.
- [20] N. Kumar, R. Kumar, and R. Caldelli, "Local moment driven pvo based reversible data hiding," *IEEE Signal Processing Letters*, vol. 28, pp. 1335–1339, 2021.
- [21] N. Ahmed, T. Natarajan, and K. Rao, "Discrete cosine transform," *IEEE Transactions on Computers*, vol. C-23, no. 1, pp. 90–93, 1974.
- [22] T. Kalker and F. Willems, "Capacity bounds and constructions for reversible data-hiding," *2002 14th International Conference on Digital Signal Processing Proceedings*, vol. 1, pp. 71–76, 2002.
- [23] W. Zhang, B. Chen, and N. Yu, "Capacity-approaching codes for reversible data hiding," *Lecture Notes in Computer Science*, pp. 255–269, 2011.
- [24] —, "Improving various reversible data hiding schemes via optimal codes for binary covers," *IEEE Transactions on Image Processing*, vol. 21, no. 6, pp. 2991–3003, 2012.
- [25] W. Zhang, X. Hu, X. Li, and Y. Nenghai, "Optimal transition probability of reversible data hiding for general distortion metrics and its applications," *IEEE Transactions on Image Processing*, vol. 24, no. 1, pp. 294–304, 2015.
- [26] W. Zhang, X. Hu, X. Li, and N. Yu, "Recursive histogram modification: Establishing equivalency between reversible data hiding and lossless data compression," *IEEE Transactions on Image Processing*, vol. 22, no. 7, pp. 2775–2785, 2013.
- [27] S.-J. Lin and W.-H. Chung, "The scalar scheme for reversible information-embedding in gray-scale signals: Capacity evaluation and code constructions," *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 4, pp. 1155–1167, 2012.
- [28] M. Xiao, X. Li, and Y. Zhao, "General distortion based reversible data hiding for binary covers," *IEEE Signal Processing Letters*, vol. 29, pp. 2537–2541, 2022.
- [29] J. J. Rissanen, "Generalized kraft inequality and arithmetic coding," *IBM Journal of Research and Development*, vol. 20, no. 3, pp. 198–203, 1976.
- [30] J. Duda, P. Korus, N. J. Gadgil, K. Tahboub, and E. J. Delp, "Image-like 2d barcodes using generalizations of the kuznetsov–tsybakov problem," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 4, pp. 691–703, 2016.
- [31] J. Duda, N. J. Gadgil, K. Tahboub, and E. J. Delp, "Generalizations of the kuznetsov–tsybakov problem for generating image-like 2d barcodes," in *2014 IEEE International Conference on Image Processing (ICIP)*, 2014, pp. 4221–4225.
- [32] S. Camtepe, J. Duda, A. Mahboubi, P. Morawiecki, S. Nepal, M. Pawłowski, and J. Pieprzyk, "Compcrypt–lightweight ANS-based compression and encryption," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3859–3873, 2021.
- [33] J. Duda, "Asymmetric numeral systems," *arXiv preprint arXiv:0902.0271*, 2009.
- [34] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [35] N. Wang, W. Yan, S.-J. Lin, and Y. Huang, "An entropy coding based on binary encoding for mixed-radix digits," *2022 Data Compression Conference (DCC)*, pp. 488–488, 2022.
- [36] Y. Jia, Z. Yin, X. Zhang, and Y. Luo, "Reversible data hiding based on reducing invalid shifting of pixels in histogram shifting," *Signal Processing*, vol. 163, pp. 238–246, 2019.



Na Wang (Member, IEEE) received the Ph.D. degree in Cyberspace Security from the University of Science and Technology of China (USTC), Hefei, China. She is currently a researcher with the School of optoelectronic information and Computer Engineering at the University of Shanghai for Science and Technology (USST). Her research focuses on entropy coding algorithms and multimedia security.



Shuxi Xu received the M.S. degree in Information Engineering from Jilin University, Jilin, China, in 2005, and is now working in East China Institute of Optoelectronic Integrated Circuits (ECIOIC), Bengbu, China, in the areas of algorithm design.



Yunghsiung S. Han (Fellow, IEEE) was born in Taipei, Taiwan, in 1962. He received B.Sc. and M.Sc. degrees in electrical engineering from the National Tsing Hua University, Hsinchu, Taiwan, in 1984 and 1986, respectively, and a Ph.D. from the School of Computer and Information Science, Syracuse University, Syracuse, NY, in 1993. From 1986 to 1988, he was a lecturer at Ming-Hsin Engineering College, Hsinchu, Taiwan. He was a teaching assistant from 1989 to 1992 and a research associate in the School of

Computer and Information Science at Syracuse University from 1992 to 1993. From 1993 to 1997, he was an Associate Professor in the Department of Electronic Engineering at Hua Fan College of Humanities and Technology, Taipei Hsien, Taiwan. He was with the Department of Computer Science and Information Engineering at National Chi Nan University, Nantou, Taiwan from 1997 to 2004. He was promoted to Professor in 1998. He was a visiting scholar in the Department of Electrical Engineering at the University of Hawaii at Manoa, HI from June to October 2001, the SUPRIA visiting research scholar in the Department of Electrical Engineering and Computer Science and CASE center at Syracuse University, NY from September 2002 to January 2004 and July 2012 to June 2013, and the visiting scholar in the Department of Electrical and Computer Engineering at the University of Texas at Austin, TX from August 2008 to June 2009. He was with the Graduate Institute of Communication Engineering at National Taipei University, Taipei, Taiwan from August 2004 to July 2010. From August 2010 to January 2017, he was Chair Professor with the Department of Electrical Engineering at the National Taiwan University of Science and Technology. From February 2017 to February 2021, he was with the School of Electrical Engineering & Intelligentization at Dongguan University of Technology, China. Now, he is with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China. He is also a Chair Professor at National Taipei University since February 2015. His research interests are in error-control coding, wireless networks, and security.

Dr. Han was a winner of the 1994 Syracuse University Doctoral Prize and a Fellow of IEEE. One of his papers won the prestigious 2013 ACM CCS Test-of-Time Award in cybersecurity.



Chuan Qin (Member, IEEE) received the B.S. degree in electronic engineering and the M.S. degree in signal and information processing from Hefei University of Technology, Anhui, China, in 2002 and 2005, respectively, and the Ph.D. degree in signal and information processing from Shanghai University, Shanghai, China, in 2008. Since 2008, he has been with the faculty of the School of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology, where he is currently a Professor.

He was with Feng Chia University, Taiwan, as a Post-Doctoral Researcher, from July 2010 to July 2012. His research interests include image processing and multimedia security. He has published over 200 articles in these research areas.



Sian-Jheng Lin (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in computer science from National Chiao Tung University, Hsinchu, Taiwan, in 2004, 2006, and 2010, respectively. From 2010 to 2014, he was a Post-Doctoral Researcher with the Research Center for Information Technology Innovation, Academia Sinica. From 2014 to 2016, he was a Post-Doctoral Researcher with the Electrical Engineering Department at King Abdullah University of Science and Technology (KAUST),

Thuwal, Saudi Arabia. From 2016 to 2021, he was a Researcher with the School of Information Science and Technology at University of Science and Technology of China (USTC), Hefei, China. He is currently an independent researcher in China. In recent years, his research focuses on the codes for storage systems and data compressions.



Shuo Shao (Member, IEEE) is an associate professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, China. He received his B.S. degree in Information Science from Southeast University, China, in 2011, his M.A.Sc degree in Electrical and Computer Engineering from McMaster University, Canada, in 2013, and his PhD degree from Texas A&M University, U.S., in 2017, respectively. His research interests are in network information theory, algebraic code, machine learning, and semantic communications.

machine learning, and semantic communications.